
Programming Techniques for Scientific Simulations

Autumn Semester 2026

Lecture Notes

Contents

1	Version Control and Git	1
2	Preprocessing, Compiling, Linking	1
3	Make	3

References

- [1] Käppeli, Roger: Vorlesungsskript PT1. ETH Zürich. 2026

Programming Techniques for Scientific Simulations

Fynn Krebsner—fkrebsner@student.ethz.ch

Autumn Semester 2026

Lec 1

1 Version Control and Git

When working on a project in a group, it is important to have a system that allows you to track changes to your code and collaborate with others. Version control systems (VCS) are tools that help manage changes to source code over time. They can be local on one computer, centralized, with one centralized repository, or decentralized, with many repositories that can be synced. An example for the last one is git, which we will discuss in this course.

These tools can be used via command line tools, GUIs or via editor plugins. We'll focus on the CLI tools. The general idea is that you have your files and a version database. Every time you are happy with your changes, you can **COMMIT** them with a short description describing the changes (**COMMIT MESSAGE**).

Version control is also useful for working in a group. For example Alice and Bob are working on a project. Alice makes some changes, commits and **PUSHes** them to a remote repository. Later on, Bob can **PULL** the changes from the remote repository and continue working on the project.

Usually however, both peoples work together. Alice commits and pushes her changes, while Bob is still working. He commits his new version and tries to push it. However, this time, the push can fail, because Alice's changes are not yet in Bob's local repository. This is a **CONFLICT**. We also say that the histories have diverged. What Bob needs to do is **MERGING**. Bob inspects both versions and decides how to combine them. After that, he can commit the merged version and push it to the remote repository.

In git, we can init a repository with `git init`. This creates a new repository in the current directory. The information for this is stored in a hidden directory called `.git`. With `git status` we can check the status of our repository. It will show us which files have been modified, added or deleted. We can add files to the staging area with `git add <file>`. This means that we are ready to commit these changes. Finally, we can commit the changes with `git commit -m "commit message"`. The commit message should be a short description of the changes made.

The **STAGING AREA** exists, so you have some freedom to decide which changes you want to commit. You can add some files to the staging area, but not others. This is useful if you are working on multiple features at the same time and want to commit them separately. Using `git diff` we can see the changes between the working directory and the staging area. With `git diff --staged` we can see the changes between the staging area and the last commit.

Usually in a repository we only want the source code and not some artifacts or executables. We can tell git to ignore

certain files by creating a file called `.gitignore`. In this file we can specify patterns for files that should be ignored by git. For example, we can ignore all files with the extension `.log` by adding the line `*.log` to the `.gitignore` file.

Sometimes it could be useful to work on two different features. In this case we can use **BRANCHES**. A branch is a separate line of development. We can create a new branch with `git branch <branch name>`. We can switch to a different branch with `git checkout <branch name>`. After having made some changes and committed them, we can switch back to the main branch and notice, there are no changes. Once we are happy with the new branch, we can merge it back into the main branch with `git merge <branch name>`. If we are done, we can delete the branch with `git branch -d <branch name>`.

If there is a merge conflict, git will mark the files that have conflicts and we need to resolve them manually. After resolving the conflicts, we can add the files to the staging area and commit the changes.

Lec 2

2 Preprocessing, Compiling, Linking

A compiler is a program that translates source code into machine code. We will mainly use GNU and Clang. When compiling a program, the **PREPROCESSOR** is the first step. It manipulates the source code. For this all commands start with a `#`. For this, we can define macros with `#define`. For example, we can define a macro `PI` as follows:

```
#define PI 3.14159265358979323846
```

The convention for macros is to use all uppercase letters. With this one can distinguish between macros and functions defined in the source code. Macros can also have arguments. For example

```
#define SUM(A,B) A + B
```

In the console this can also be done with the flag `-D`. For example, we can define a macro `PI` as follows:

```
g++ -DPI=3.14159265358979323846 main.cpp
```

We can also undefine macros with `#undef` and in the console with the flag `-U`. For example, we can undefine a macro `PI` as follows:

```
#undef PI
```

To look at only the preprocessed output is to do

```
g++ -E main.cpp
```

The preprocessor also allows us to include other files with `#include`. For example, we can include the header file `iostream`.

As a warning, the preprocessor does not check for syntax errors. It simply replaces the macros and includes the files. Therefore, it is important to be careful when using macros and includes.

For conditional compilation, we can use `#ifdef`.

```
1 #ifdef PI
2     std::cout << "PI is defined" << std::endl;
3 #else
4     std::cout << "PI is not defined" << std::endl;
5 #endif
```

When including files, we can include them with brackets or with quotes. The brackets are used for system headers, while the quotes are used for user-defined headers. With the `-I` flag, we can tell the compiler where to look for the header files.

The next step is the **COMPILER**. It turns the preprocessed code into object code. We can view the object code with the `-S` flag. For example, we can compile a file `main.cpp` into an object file `main.o` as follows:

```
1 g++ -c main.cpp -o main.o
```

Furthermore we can change the optimization level with the `-O` flag. The optimization levels are from 0 to 3. The higher the optimization level, the more time it takes to compile, but the faster the program will run. The default optimization level is 0. For example, we can compile a file `main.cpp` with optimization level 2 as follows:

```
1 g++ -O2 main.cpp -o main
```

After that, the assembler takes the object code and turns it into machine code. The machine code is then linked with other object files and libraries to create an executable file.

To see how linking works, let us consider some code refactoring. In general, Programs can be split into several files, that can be compiled separately and finally lined together. For this in a `.hpp` file we put in the function declarations and in a `.cpp` file we put in the function definitions. For example, we can have a file `math.hpp` with the following content:

```
1 double add(double, double);
2 double subtract(double, double);
```

And a file `math.cpp` with the following content:

```
1 #include "math.hpp"
2 double add(double a, double b) {
3     return a + b;
4 }
5 double subtract(double a, double b) {
6     return a - b;
7 }
```

Usually we include the `hpp` file instead of the `cpp` file. This is done to enforce the consistency between the declaration and the definition. If we include the `cpp` file, we can have a situation where the declaration and the definition are not consistent. For example, if we change the declaration in the `hpp` file, but forget to change the definition in the `cpp` file, we will get a linker error.

To compile all files separately we compile the files with the `-c` flag. For example, we can compile the files `math.cpp` and `main.cpp` as follows:

```
1 g++ -c math.cpp -o math.o
2 g++ -c main.cpp -o main.o
```

After that we can link them with

```
1 g++ math.o main.o -o main
```

One thing to keep in mind is that when including headers, we should use include guards to prevent multiple inclusions of the same header file. This can be done with the following pattern:

```
1 #ifndef MATH_HPP
2 #define MATH_HPP
3 double add(double, double);
4 double subtract(double, double);
5 #endif
```

Alternatively one can also use `#pragma once` at the top of the header file. This is a non-standard but widely supported way to prevent multiple inclusions of the same header file.

```
1 #pragma once
2 double add(double, double);
3 double subtract(double, double);
```

One useful macro might be `NDEBUG`. If this macro is defined, the `assert` macro will not check for the condition and will not terminate the program if the condition is false. This can be useful for performance reasons, as the `assert` macro can add overhead to the program. However, it is important to note that this can also lead to undefined behavior if the condition is false and the program continues to run.

Libraries come in two kinds. First, static libraries aka archives. When linking, only the used functions are linked into the executable. Second, dynamic libraries aka shared objects. When linking, all functions are linked into the executable. The dynamic libraries are loaded at runtime. This means that if a function is not used, it will not be loaded into memory. This can save memory and reduce the size of the executable. However, it also means that if a function is used, it will be loaded into memory and will take up space.

We can pack the `*.o` files into a static library with the `ar` command. For example, we can create a static library `libmath.a` from the object files `math.o` and `main.o` as follows:

```
1 ar -crs libmath.a math.o main.o
```

Important the `lib` prefix is required for static libraries. Otherwise, the linker will not be able to understand it.

For shared libraries, we use

```
1 c++ -shared -fPIC -o libmath.so math.o main.o
```

The advantage of shared libraries is that the executables are shared between different programs. Thus it might suffice to only have one copy of `cout` for example.

We now compile and link the program with the shared library as follows:

```
1 c++ -c -Ilib main.cpp
2 c++ -o main main.o -Llib -lmath
```

Here `-L` specifies the path to the library and `-l` specifies the name of the library. The linker will look for a file named `libmath.so` in the specified path. `-I` specifies the path to the header files. The linker will look for a file named `math.hpp` in the specified path.

Warning, the order in which libraries are specified is important. The linker will look for the libraries in the order they are specified. If a library is specified before another library that depends on it, the linker will not be able to find the symbols in the first library and will throw an error.

After having made a library, we can document it. For this we should document:

- Synopsis
- Semantics
- Template Parameters
- Parameters
- Return Value
- Precons
- Postcons
- Dependencies
- Exception Guarantees
- Complexity
- References

Unfortunately, after a while, it might become tedious to compile and link many different files. We need to keep track of who depends on whom. Or, we could use build systems.

3 Make

Make is a build automation tool. It is probably the most widely used build tool. Make builds targets based on so-called build files. Usually, the build file is named `Makefile`. Comments are made with hashtags. A simple program might be

```
1 hello:
2     echo Hello, World!
```

Another useful thing might be a dry run to see what would be done without actually doing it. This can be done with the `-n` flag.

For our math library, we can create a Makefile as follows:

```
1 math.o: math.cpp math.hpp
2     c++ -c math.cpp -o math.o
3
4 main.o: main.cpp math.hpp
5     c++ -c main.cpp -o main.o
6
7 main: math.o main.o
8     c++ -o main main.o math.o
```

However, make only does the first target. What we can do is a phony target. A phony target is a target that is

not a file. It is used to group other targets together. For example, we can create a phony target `all` that builds all targets as follows:

```
1 .PHONY: all
2 all: main
```

Make here is pretty smart and will only rebuild the targets that have changed. It does this by checking the timestamps of the files. If a file has changed, it will rebuild the target that depends on it. If a file has not changed, it will not rebuild the target that depends on it.

Index

branches, 1

commit, 1

commit message, 1

Compiler, 2

conflict, 1

merging, 1

Preprocessor, 1

pull, 1

push, 1

staging area, 1